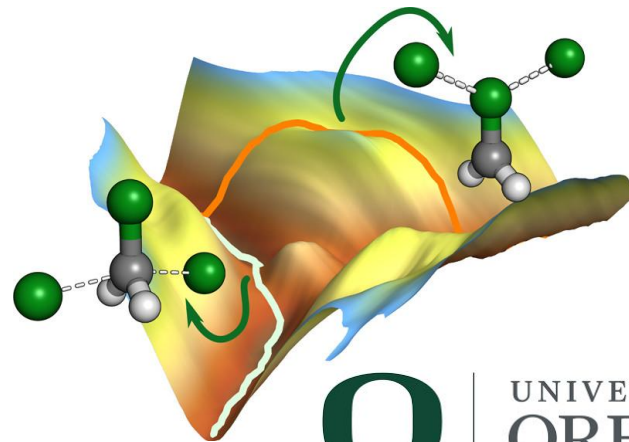
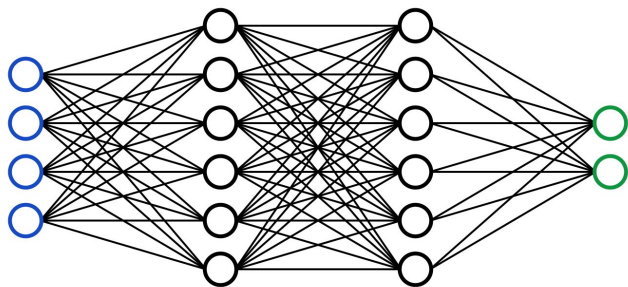


Machine Learning for Chemistry

Winter 2025

Advanced Topic 1: Convolutional Neural Networks



UNIVERSITY OF
OREGON

Instructor: Dr. Dhiman Ray

Assistant Professor, Department of Chemistry and Biochemistry

Overview of Topics

- Discrete Convolution
- Padding
- Pooling
- Convolutional Neural Networks

References:

1. **Machine Learning with Pytorch and Scikit-Learn** by Raschka, Liu, and Mirjalili (Chapter 14)

Convolution

Identification of pattern irrespective of translation, rotation, or inversion.

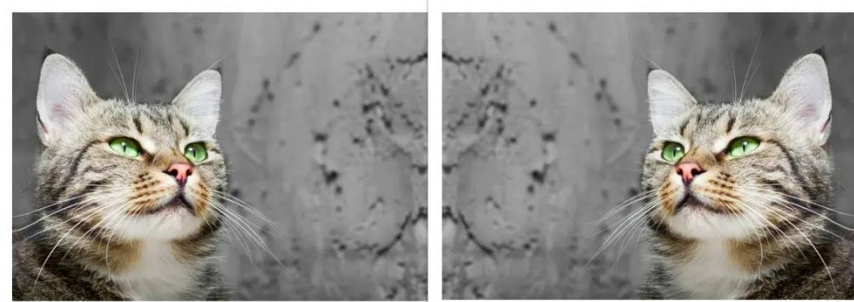
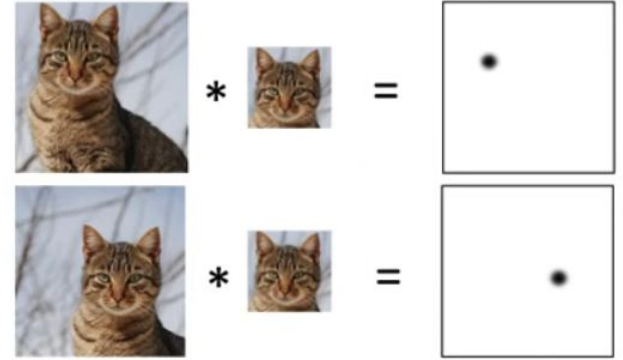
We should be able to classify these images as cat.



Convolution

Identification of pattern irrespective of translation, rotation, or inversion.

We should be able to classify these images as cat.



Convolution

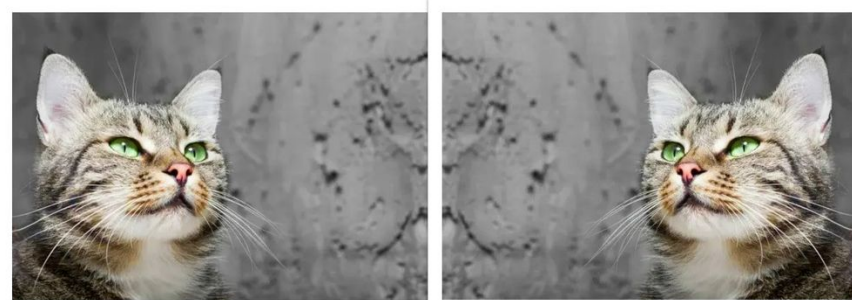
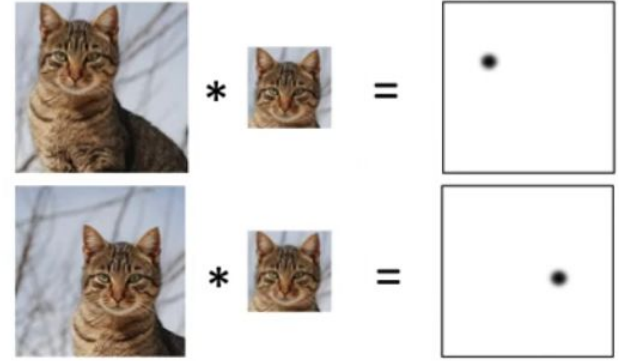
Identification of pattern irrespective of translation, rotation, or inversion.

We should be able to classify these images as cat.

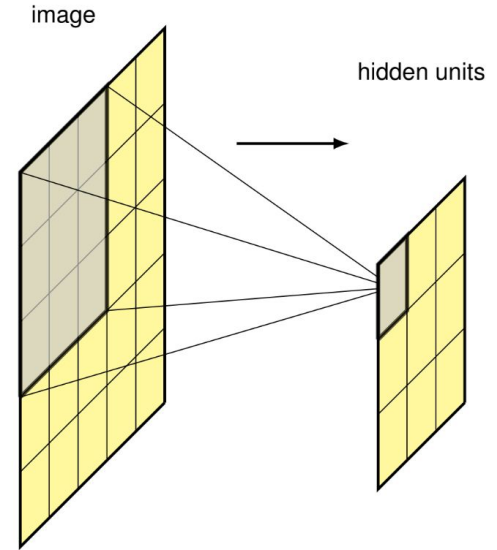
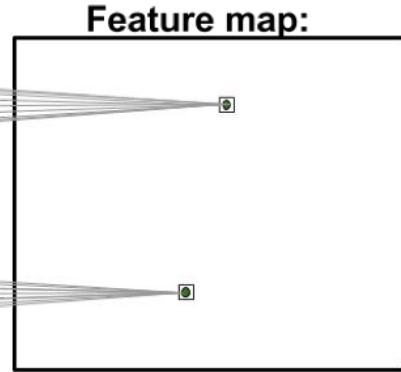
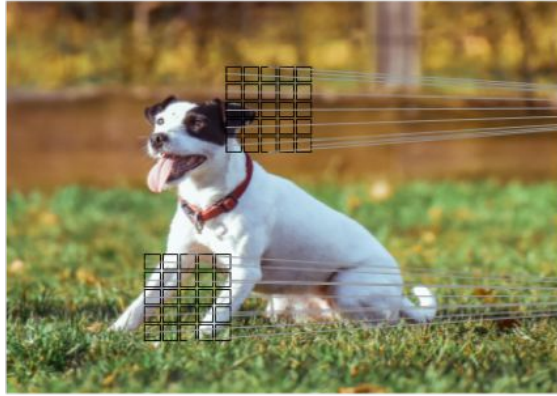
$$\mathbf{y} = \mathbf{x} * \mathbf{w} \rightarrow y[i] = \sum_{k=-\infty}^{+\infty} x[i - k] w[k]$$

Where x is the original input, w is the filter, and y is the convolved input.

Convolution layers extract low-level features from raw data, and the later layers of a neural network use these features to predict a continuous target value or class label.



Convolution

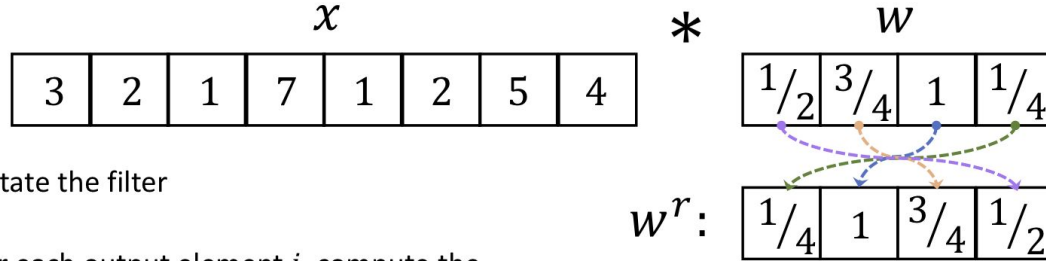


Convolutional Neural Network (CNN) computes feature maps from an input image, where each element comes from a local patch of pixels in the input image.

This is different from connecting to the whole input image as in standard NNs.

Replacing a conventional, fully connected MLP with a convolution layer decreases the number of weights (parameters) in the network, and improves the ability to capture salient features.

Discrete Convolution in 1D



$y = [7 \ 9 \ 8]$

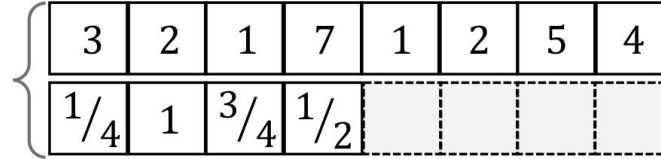
Step 1: Rotate the filter

Step 2: For each output element i , compute the dot-product $x[i:i+4] \cdot w^r$

(move the filter two cells)

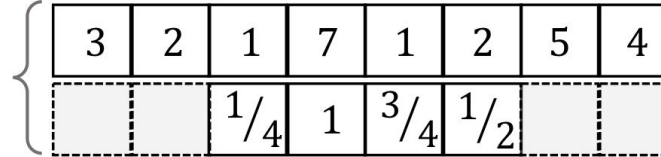
$$y[0] = 3 \times \frac{1}{4} + 2 \times 1 + 1 \times \frac{3}{4} + 7 \times \frac{1}{2}$$

$$\rightarrow y[0] = 7$$



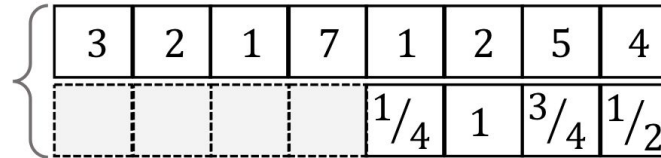
$$y[1] = 1 \times \frac{1}{4} + 7 \times 1 + 1 \times \frac{3}{4} + 2 \times \frac{1}{2}$$

$$\rightarrow y[1] = 9$$



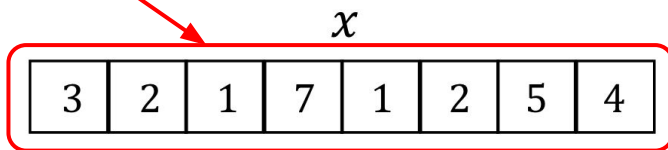
$$y[2] = 1 \times \frac{1}{4} + 2 \times 1 + 5 \times \frac{3}{4} + 4 \times \frac{1}{2}$$

$$\rightarrow y[2] = 8$$

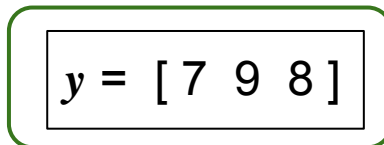
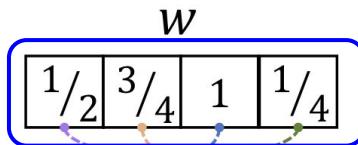


Discrete Convolution in 1D

Input



*



Step 1: Rotate the filter



Step 2: For each output element i , compute the dot-product $x[i : i + 4] \cdot w^r$

(move the filter two cells)

Stride = 2

$$y[0] = 3 \times \frac{1}{4} + 2 \times 1 + 1 \times \frac{3}{4} + 7 \times \frac{1}{2}$$

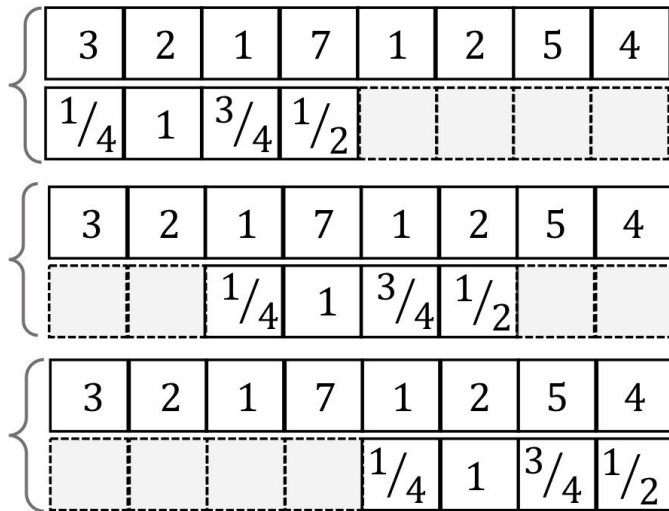
$$\rightarrow y[0] = 7$$

$$y[1] = 1 \times \frac{1}{4} + 7 \times 1 + 1 \times \frac{3}{4} + 2 \times \frac{1}{2}$$

$$\rightarrow y[1] = 9$$

$$y[2] = 1 \times \frac{1}{4} + 2 \times 1 + 5 \times \frac{3}{4} + 4 \times \frac{1}{2}$$

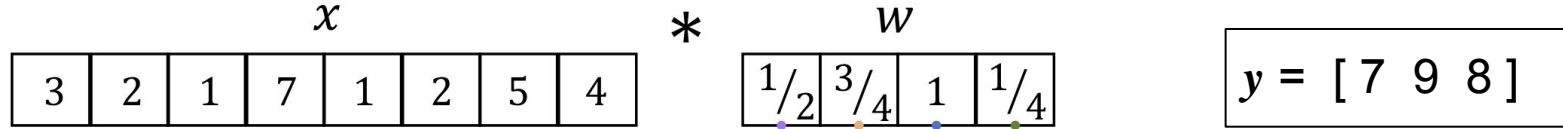
$$\rightarrow y[2] = 8$$



Filter
(kernel)

Feature

Discrete Convolution in 1D



Step 1: Rotate the filter



Read only once

Step 2: For each output element i , compute the dot-product $x[i : i + 4] \cdot w^r$

(move the filter two cells)

$$y[0] = 3 \times \frac{1}{4} + 2 \times 1 + 1 \times \frac{3}{4} + 7 \times \frac{1}{2}$$

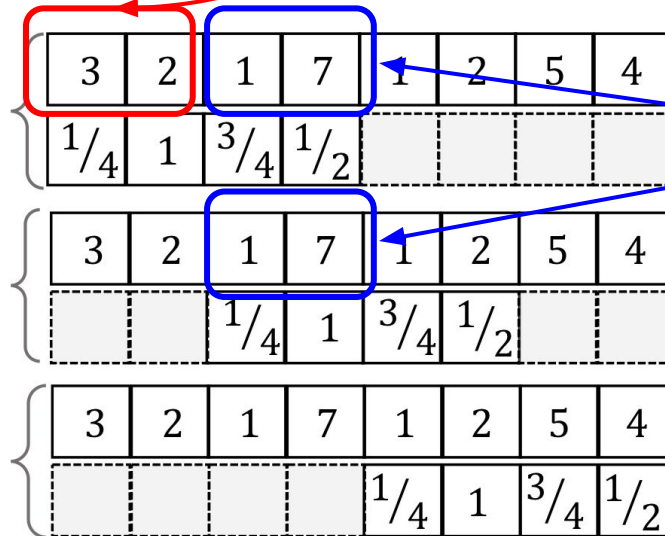
$$\rightarrow y[0] = 7$$

$$y[1] = 1 \times \frac{1}{4} + 7 \times 1 + 1 \times \frac{3}{4} + 2 \times \frac{1}{2}$$

$$\rightarrow y[1] = 9$$

$$y[2] = 1 \times \frac{1}{4} + 2 \times 1 + 5 \times \frac{3}{4} + 4 \times \frac{1}{2}$$

$$\rightarrow y[2] = 8$$



Read twice

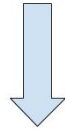
Does not give equal weight to each pixel.

Use Padding

Padding

Original x :

3	2	1	7	1	2	5	4
---	---	---	---	---	---	---	---



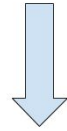
Padding
with $p=2$

0	0	3	2	1	7	1	2	5	4	0	0
---	---	---	---	---	---	---	---	---	---	---	---

Padding

Original x :

3	2	1	7	1	2	5	4
---	---	---	---	---	---	---	---



Padding
with $p=2$

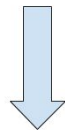
0	0	3	2	1	7	1	2	5	4	0	0
---	---	---	---	---	---	---	---	---	---	---	---



Padding

Original $\mathbf{x}$:

3	2	1	7	1	2	5	4
---	---	---	---	---	---	---	---



Padding
with $p=2$

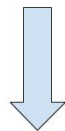
0	0	3	2	1	7	1	2	5	4	0	0
---	---	---	---	---	---	---	---	---	---	---	---



Padding

Original x :

3	2	1	7	1	2	5	4
---	---	---	---	---	---	---	---



Padding
with $p=2$

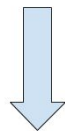
0	0	3	2	1	7	1	2	5	4	0	0
---	---	---	---	---	---	---	---	---	---	---	---



Padding

Original x :

3	2	1	7	1	2	5	4
---	---	---	---	---	---	---	---



Padding
with $p=2$

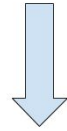
0	0	3	2	1	7	1	2	5	4	0	0
---	---	---	---	---	---	---	---	---	---	---	---



Padding

Original x :

3	2	1	7	1	2	5	4
---	---	---	---	---	---	---	---



Padding
with $p=2$

0	0	3	2	1	7	1	2	5	4	0	0
---	---	---	---	---	---	---	---	---	---	---	---

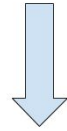


Every feature is read
2 times

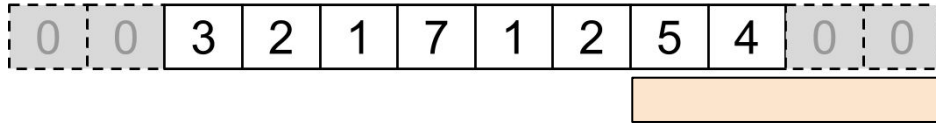
Padding

Original x :

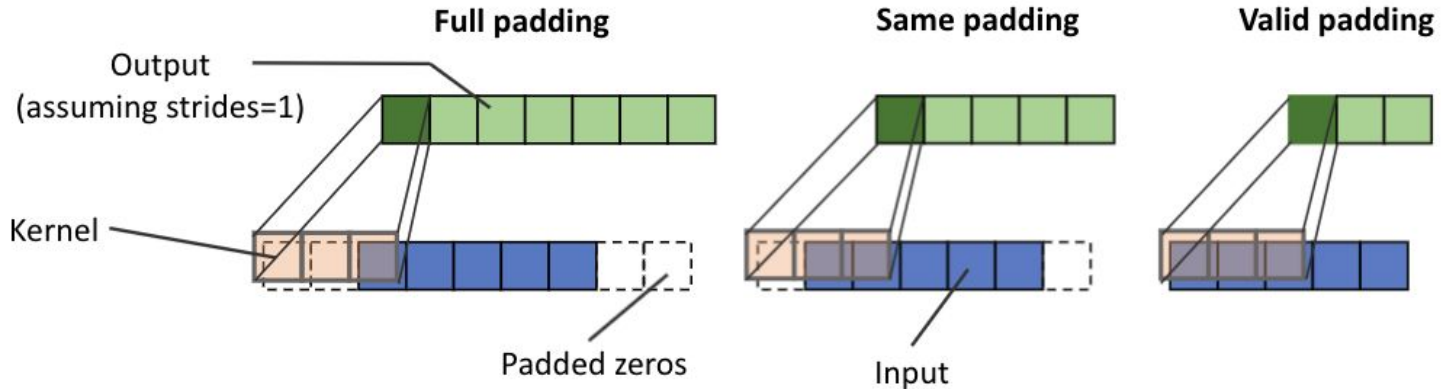
3	2	1	7	1	2	5	4
---	---	---	---	---	---	---	---



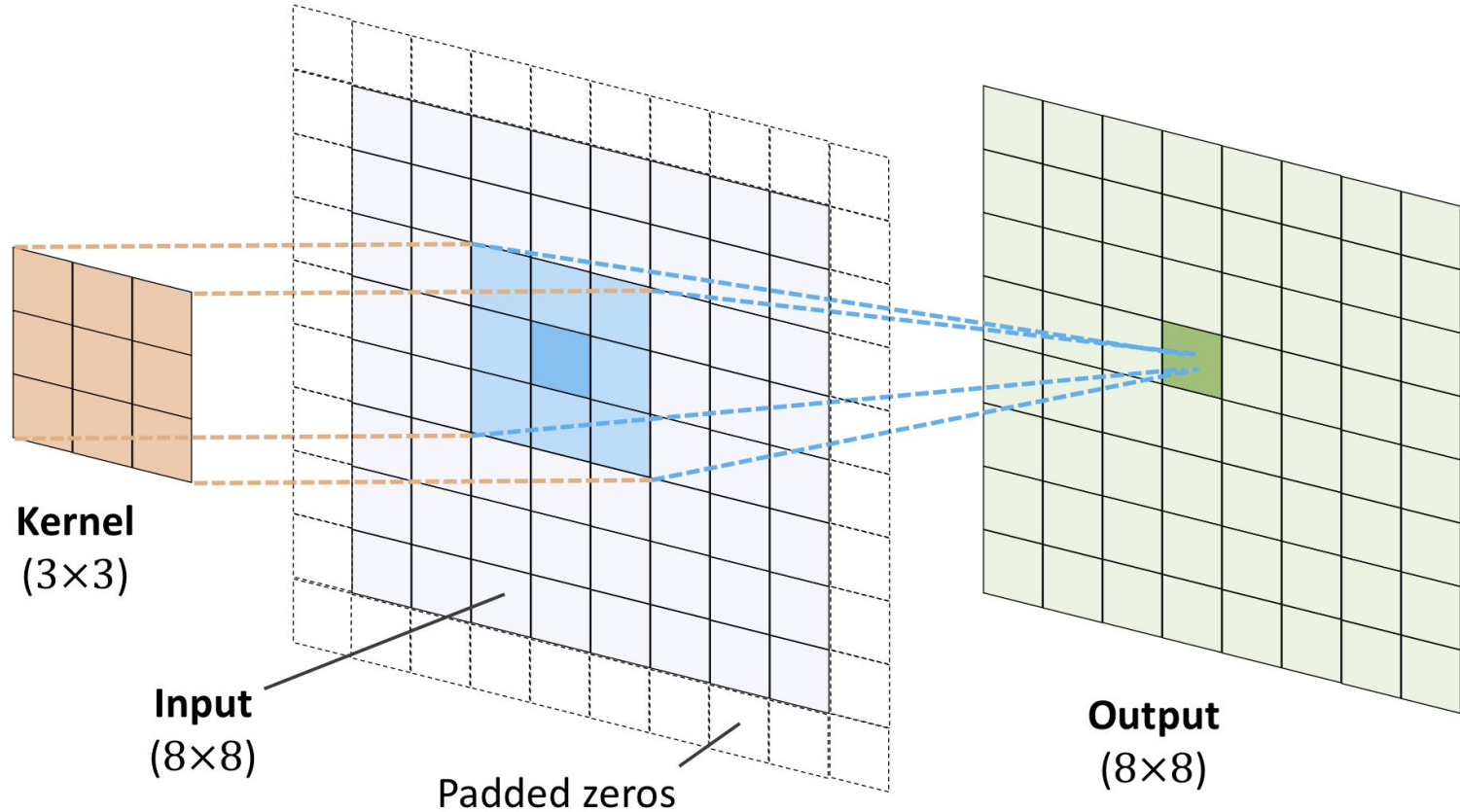
Padding
with $p=2$



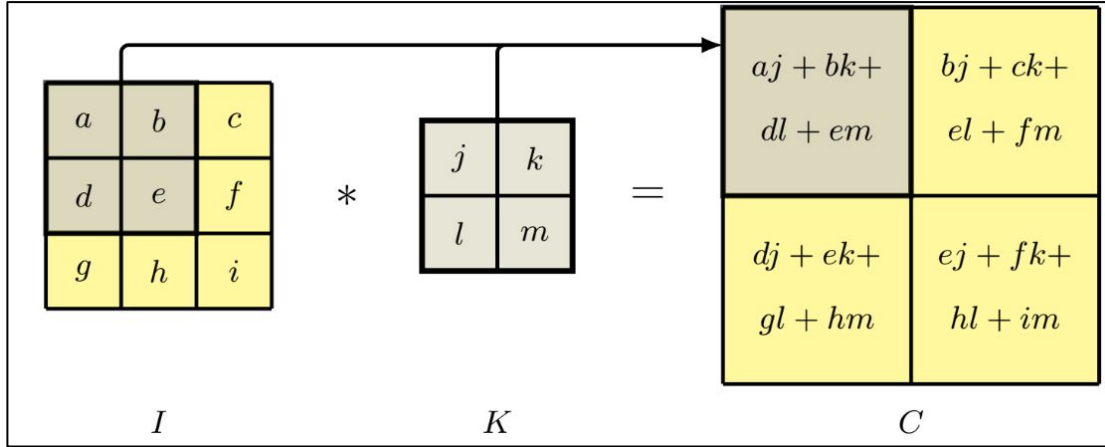
Every feature is read
2 times



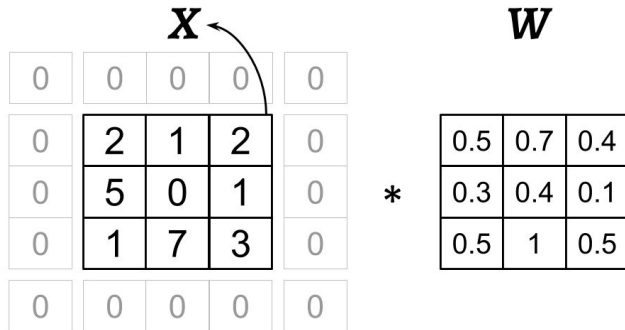
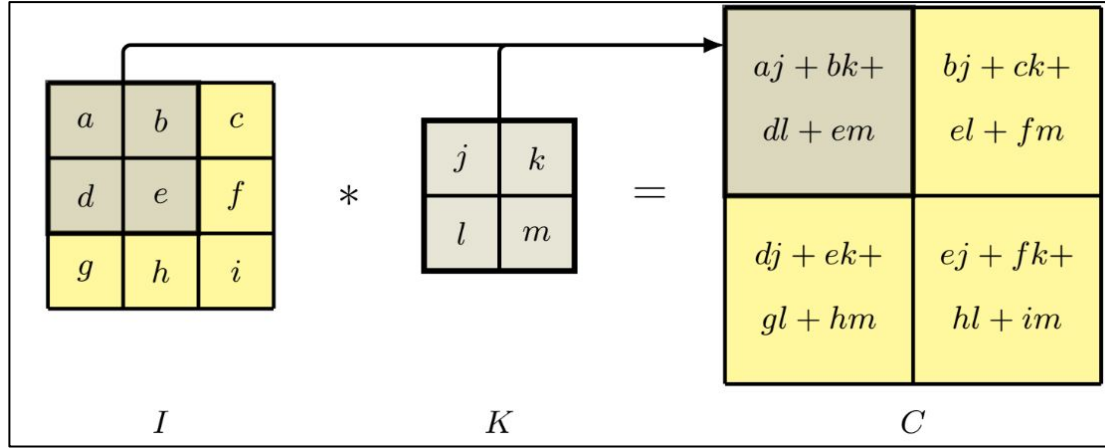
Discrete Convolution in 2D



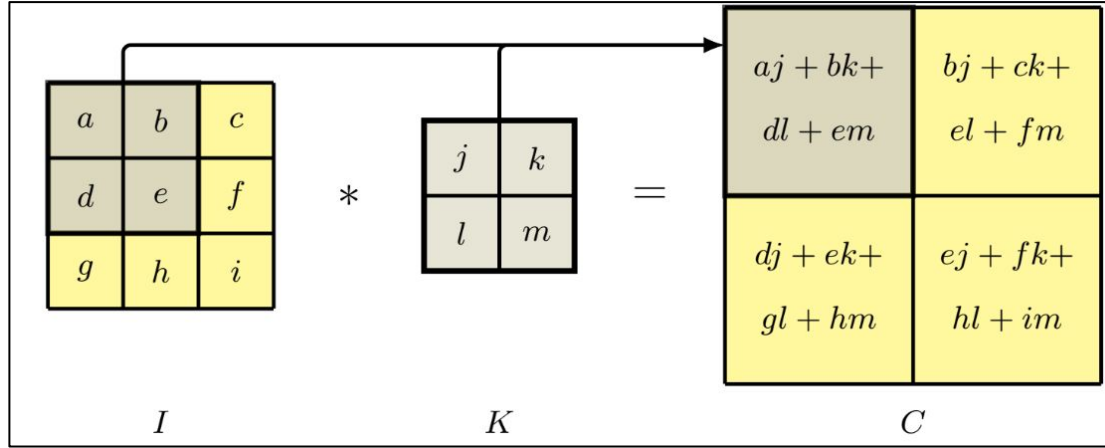
Discrete Convolution in 2D



Discrete Convolution in 2D



Discrete Convolution in 2D



X

0	0	0	0	0
0	2	1	2	0
0	5	0	1	0
0	1	7	3	0
0	0	0	0	0

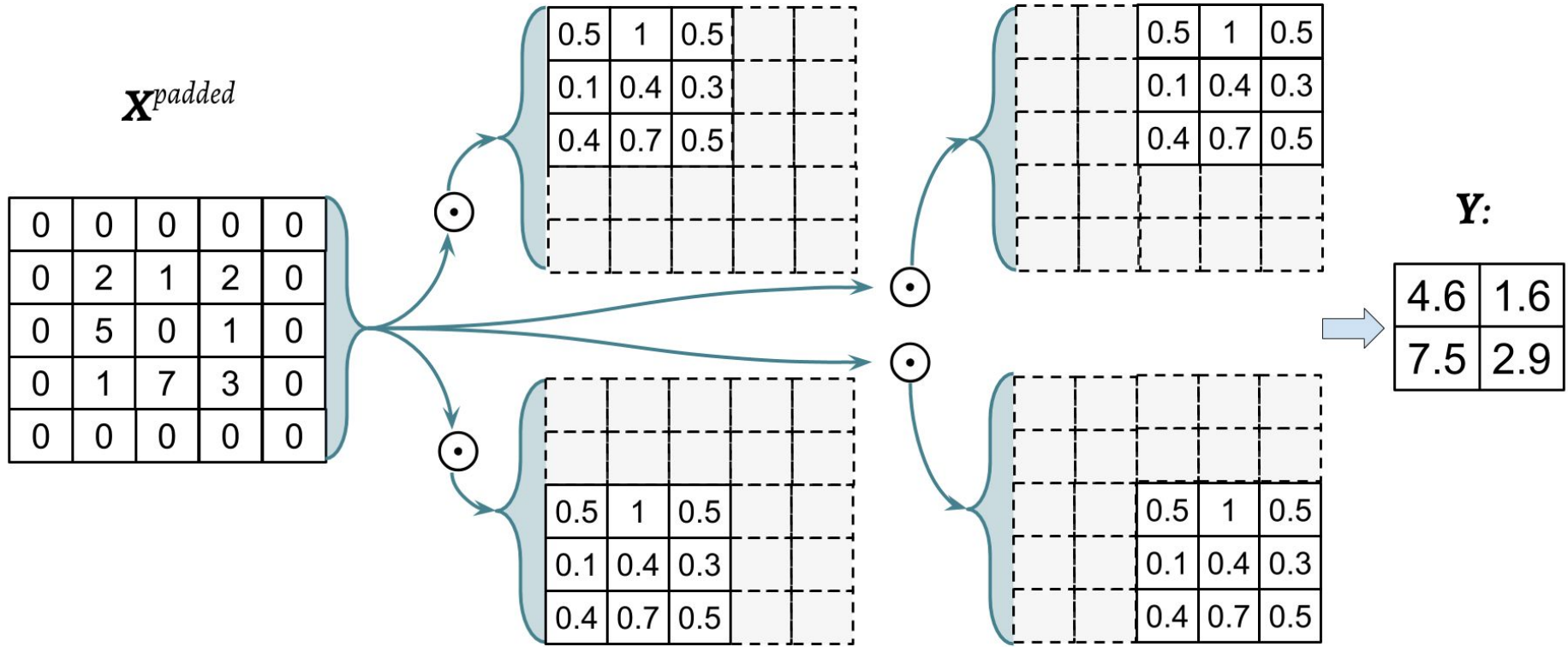
W

0.5	0.7	0.4
0.3	0.4	0.1
0.5	1	0.5

$$W^r = \begin{bmatrix} 0.5 & 1 & 0.5 \\ 0.1 & 0.4 & 0.3 \\ 0.4 & 0.7 & 0.5 \end{bmatrix}$$

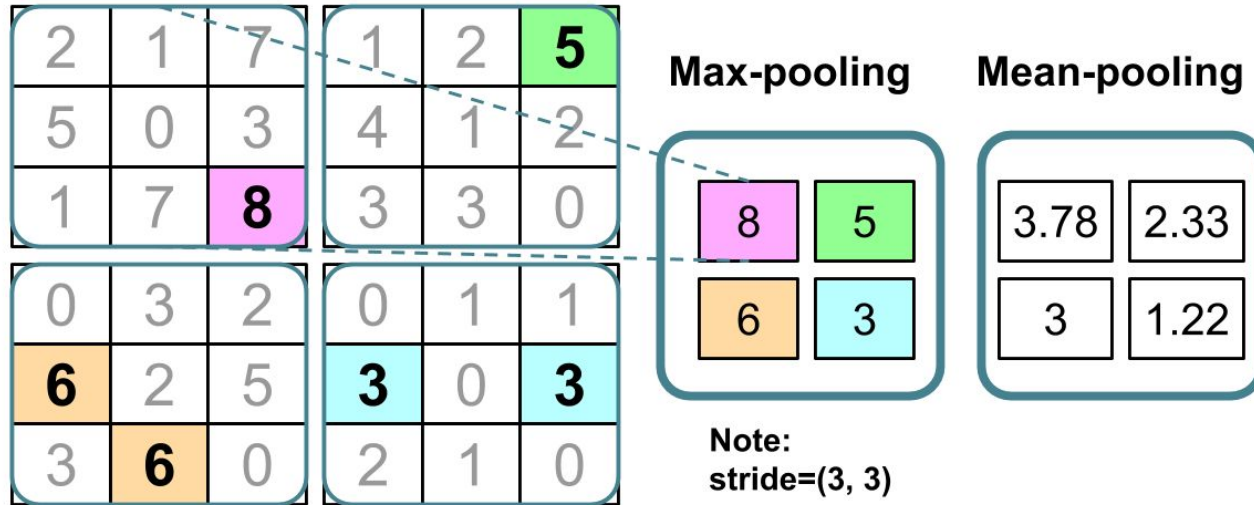
Note that this rotation is not the same as the transpose matrix. To get the rotated filter in NumPy, we can write `W_rot=W[::-1,:-1]`

Discrete Convolution in 2D



Subsampling Layers: Pooling

Pooling ($P_{3 \times 3}$)



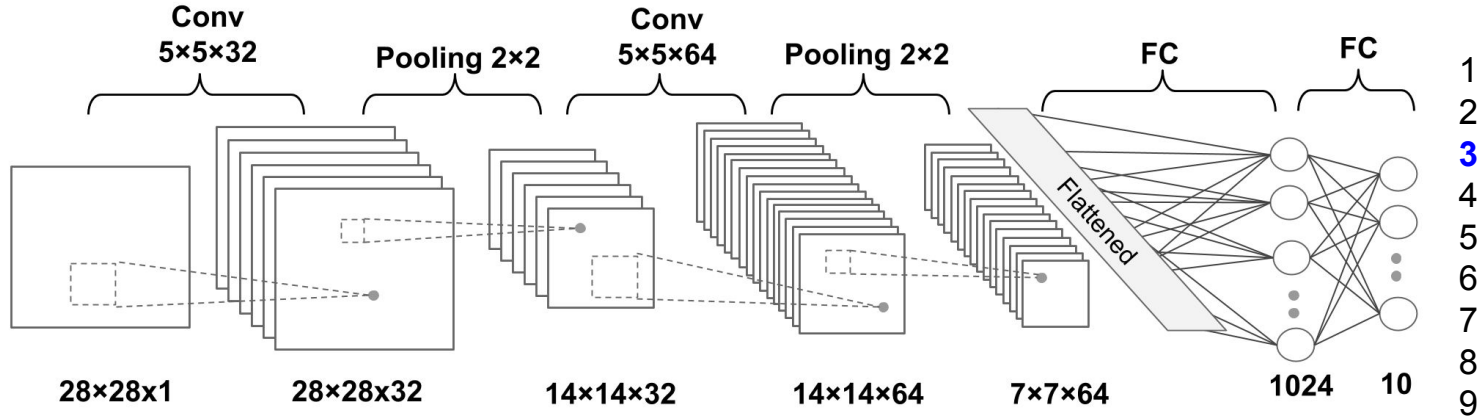
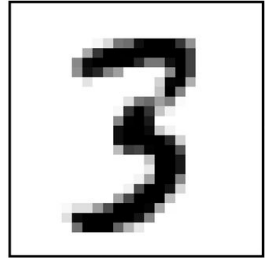
The pooling layer is usually denoted by $P_{n_1 \times n_2}$. Here, the subscript determines the size of the neighborhood (the number of adjacent pixels in each dimension) where the max or mean operation is performed.

Advantages of Pooling

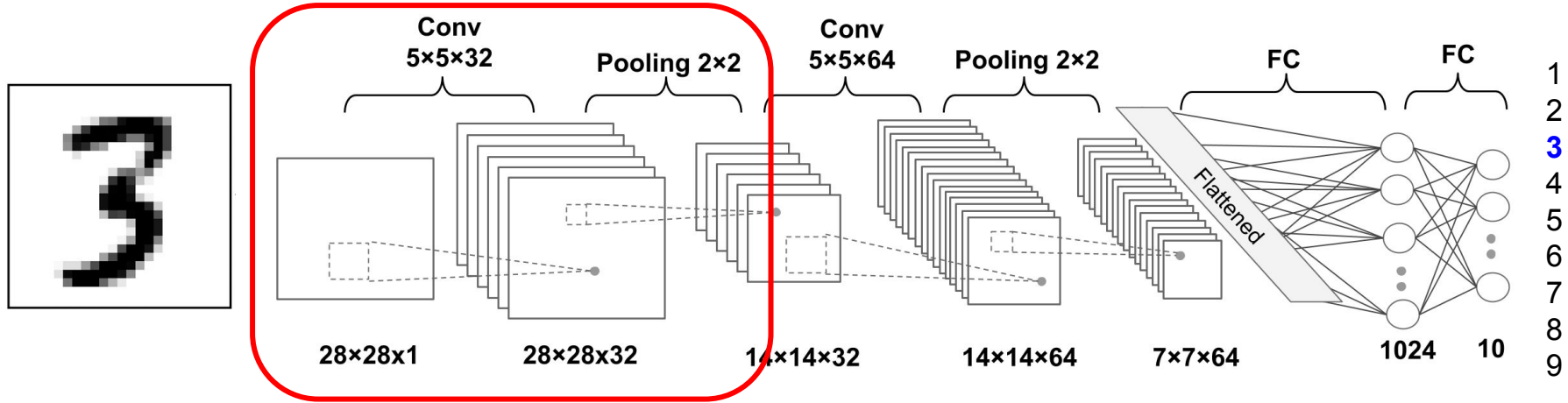
- Max pooling introduces a local invariance. **Small changes in a local neighborhood do not change the result of max-pooling.** Therefore, it helps with generating features that are more robust to noise in the input data. For example, max-pooling of two different input matrices (right) results in the same output
- Pooling decreases the size of features**, which results in higher computational efficiency. Reducing the number of features may reduce the degree of overfitting as well.

$$\begin{array}{l} X_1 = \begin{bmatrix} 10 & 255 & 125 & 0 & 170 & 100 \\ 70 & 255 & 105 & 25 & 25 & 70 \\ 255 & 0 & 150 & 0 & 10 & 10 \\ 0 & 255 & 10 & 10 & 150 & 20 \\ 70 & 15 & 200 & 100 & 95 & 0 \\ 35 & 25 & 100 & 20 & 0 & 60 \end{bmatrix} \\ \\ X_2 = \begin{bmatrix} 100 & 100 & 100 & 50 & 100 & 50 \\ 95 & 255 & 100 & 125 & 125 & 170 \\ 80 & 40 & 10 & 10 & 125 & 150 \\ 255 & 30 & 150 & 20 & 120 & 125 \\ 30 & 30 & 150 & 100 & 70 & 70 \\ 70 & 30 & 100 & 200 & 70 & 95 \end{bmatrix} \end{array} \left. \vphantom{\begin{array}{l} X_1 \\ X_2 \end{array}} \right\} \xrightarrow{\text{max pooling } P_{2 \times 2}} \begin{bmatrix} 255 & 125 & 170 \\ 255 & 150 & 150 \\ 70 & 200 & 95 \end{bmatrix}$$

Convolutional Neural Network (CNN)



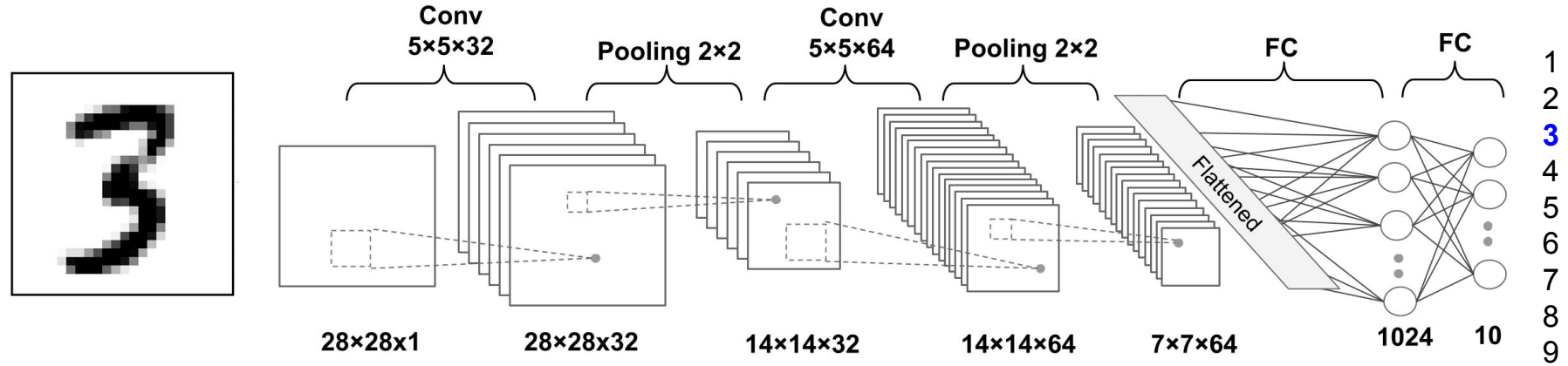
Convolutional Neural Network (CNN)



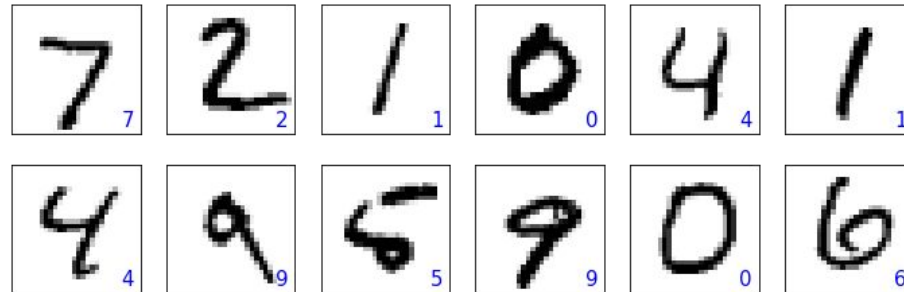
```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size=5, padding=2)
```

```
nn.MaxPool2d(kernel_size=2)
```

Convolutional Neural Network (CNN)



Output



Convolutional Neural Network (CNN)



GT: Not Smile
 $\text{Pr}(\text{Smile})=58\%$



GT: Not Smile
 $\text{Pr}(\text{Smile})=4\%$



GT: Not Smile
 $\text{Pr}(\text{Smile})=1\%$



GT: Smile
 $\text{Pr}(\text{Smile})=98\%$



GT: Not Smile
 $\text{Pr}(\text{Smile})=24\%$



GT: Not Smile
 $\text{Pr}(\text{Smile})=13\%$



GT: Not Smile
 $\text{Pr}(\text{Smile})=15\%$



GT: Smile
 $\text{Pr}(\text{Smile})=100\%$



GT: Smile
 $\text{Pr}(\text{Smile})=97\%$

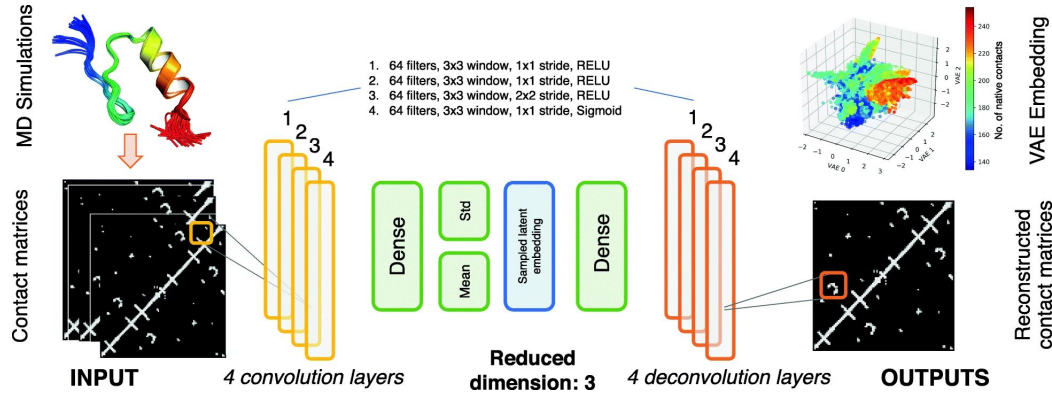


GT: Smile
 $\text{Pr}(\text{Smile})=93\%$

Smile classification on the CelebA dataset: Estimating the probability that the subject of the picture is smiling.

Example in Chapter 14: of Raschka, Liu, and Mirjalili book

Application to Molecules



Convolutional Neural Network for protein folding dynamics

Bhowmik, D., Gao, S., Young, M.T. et al. Deep clustering of protein folding simulations.

BMC Bioinformatics 19, 484 (2018).

<https://doi.org/10.1186/s12859-018-2507-5>

